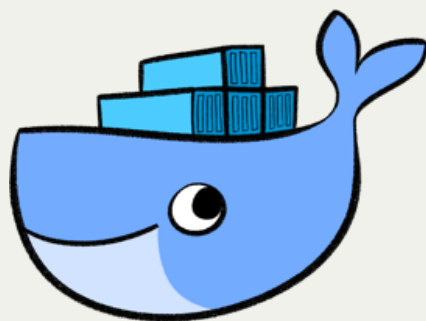


DOCKER 从入门到实践

om/static/7994cfce33b4e065.svg

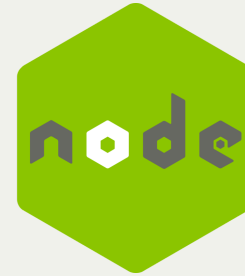


Index

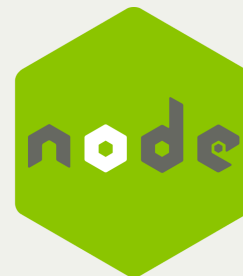
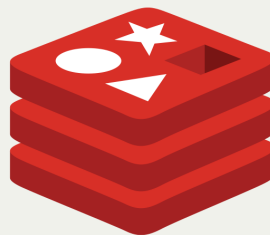
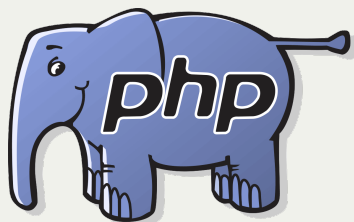
- 01 | Why Docker?
- 02 | Docker 基础
- 03 | Dockerfile
- 04 | Docker compose
- 05 | Docker 相关

WHY DOCKER ?





...



...

带来的问题

1. 不同版本同环境冲突
2. 系统未隔离相互影响
3. 大量的环境配置时间





带来的问题

1. 系统镜像特别大
2. 打包无法自动化
3. 启动及资源占用



带来的问题

1. 系统镜像特别大
2. 打包无法自动化
3. 启动及资源占用



带来的问题

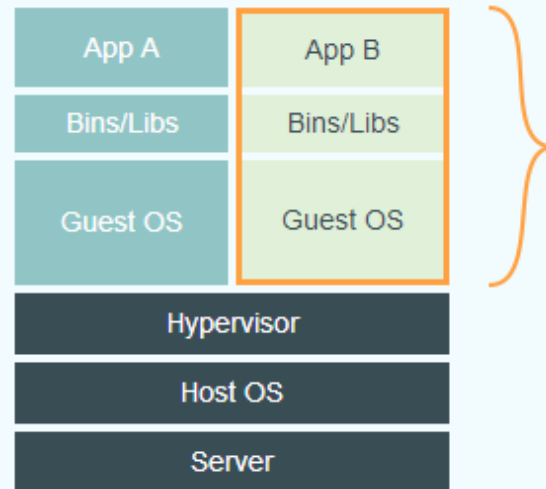
1. 系统镜像特别大
- ~~2. 打包无法自动化~~
3. 启动及资源占用



优点

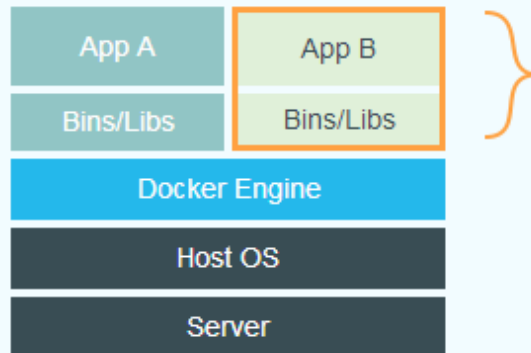
1. 体积小巧
2. 启动快速资源占用低
3. 沙箱机制环境隔离
3. 打包运行 CLI 自动化
4. Docker Hub

VM VS. DOCKER



Virtual Machines

Each virtualized application includes not only the application - which may be only 10s of MB - and the necessary binaries and libraries, but also an entire guest operating system - which may weigh 10s of GB.



Docker

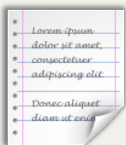
The Docker Engine container comprises just the application and its dependencies. It runs as an isolated process in userspace on the host operating system, sharing the kernel with other containers. Thus, it enjoys the resource isolation and allocation benefits of VMs but is much more portable and efficient.

DOCKER 基础



DOCKER 安装





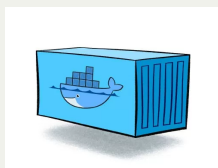
Dockerfile



镜像 (Image)



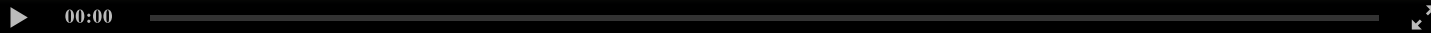
仓库 (Repository)



容器 (Container)

\$ docker run [组织名称]/<镜像名称>:[镜像标签]

```
# lizheming @ Coders-Mac in ~ [13:26:01]  
$ docker run hello-world  
Unable to find image 'hello-world:latest' locally
```



Recorded with [asciinema](#)

<https://asciinema.org/a/170462/embed?>

DOCKER HUB

<http://hub.docker.com>

```
$ docker search nginx
```

Counts?

As of early this week, there were just under 400K public images on Docker Hub (376663 to be exact). This is growing by roughly 4–5K each week as new images are created.

via: [《How Many Public Images are there on Docker Hub?》](#)

如何运行一个 NGINX 容器?

```
$ docker run \  
-d \  
--rm \  
-p 8080:80 \  
-v "$PWD/workspace":/var/www/hello.world \  
-v "$PWD/hello.world.conf":/etc/nginx/conf.d/hello.world.conf \  
nginx
```

```
// hello.world.conf  
server {  
    listen    80;  
    server_name    hello.world;  
    root /var/www/hello.world;  
    index     index.html index.htm index.php;  
}
```

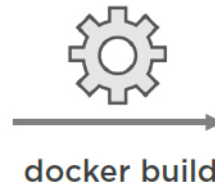
FBI WARNING

千万不要在容器实例中存储内容，实例销毁时实例内的所有内容都会被销毁,下次启动的时候又是全新的实例，内容不会保存下来。如果需要存储服务需要使用挂载卷或者外部存储服务。

DOCKERFILE



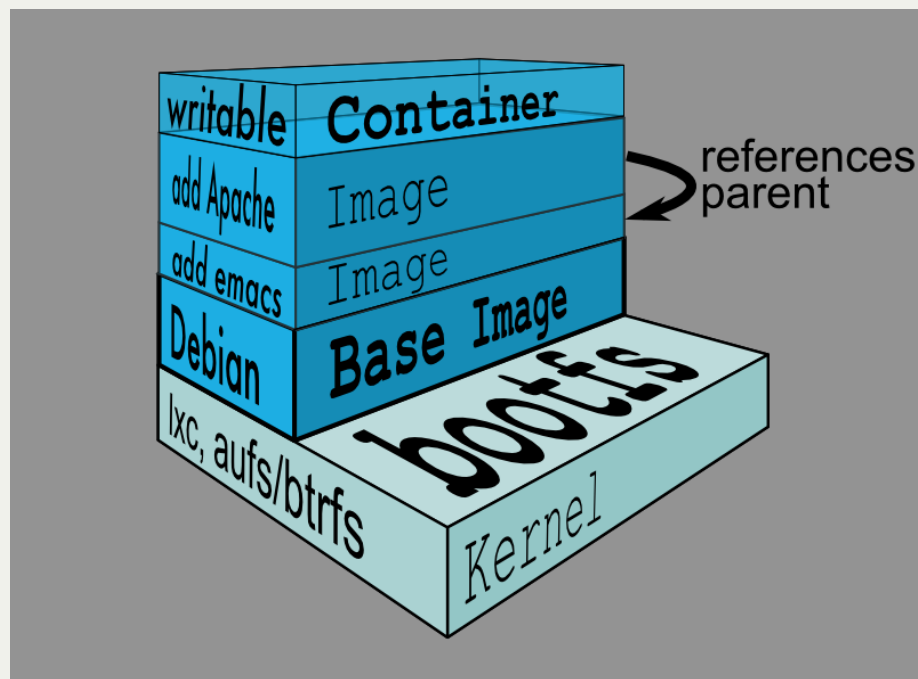
Dockerfile



Docker Image

DOCKERFILE 好处

1. 版本管理，自动化部署
2. 可增量更新，复用镜像块，镜像体积小



DOCKERFILE 规则

1. 使用#来注释
2. **FROM** 指令告诉 Docker 使用哪个镜像作为基础
3. **RUN** 开头的指令会在创建中运行，比如安装一个软件包
4. **COPY** 指令将文件复制进镜像中
5. **WORKDIR** 指定工作目录
6. **CMD/ENTRYPOINT** 容器启动执行命令

NODE DOCKERFILE 示例

```
FROM mhart/alpine-node:8.9.3

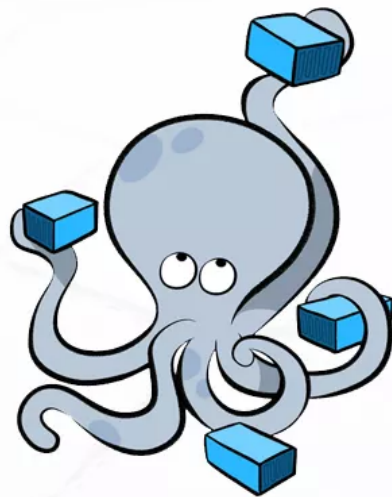
LABEL maintainer="lizheming <i@imnerd.org>" \
  org.label-schema.name="Drone Wechat Notification" \
  org.label-schema.vendor="lizheming" \
  org.label-schema.schema-version="1.1.0"

WORKDIR /wechat
COPY package.json /wechat/package.json
RUN npm install --production --registry=https://registry.npm.taobao.org

COPY index.js /wechat/index.js
ENTRYPOINT [ "node", "/wechat/index.js" ]
```

\$ docker build lizheming/drone-wechat:latest

DOCKER COMPOSE



```
php:
  build: php
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./php/www:/var/www/html
  links:
    - db
```

\$ docker-compose up

DOCKER COMPOSE 安装

抛开脚本，基于 yaml 配置的轻量级容器编排技术

安装	<code>\$ pip install docker-compose</code>
启动	<code>\$ docker-compose -f docker-compose.yml up</code>
停止	<code>\$ docker-compose stop</code>

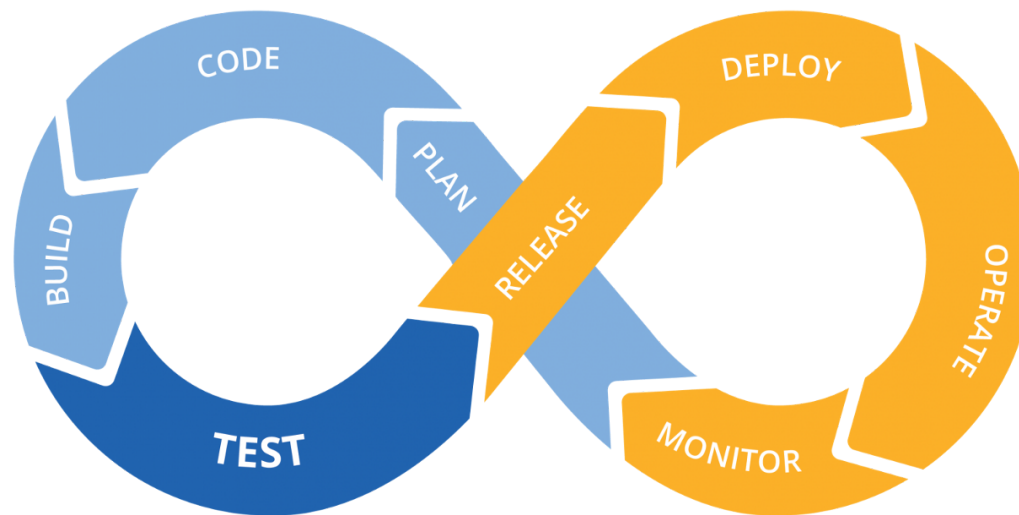
DOCKER-COMPOSE.YML

```
version: "2"
services:
  nginx:
    depends_on:
      - "php"
    image: "nginx:latest"
    volumes:
      - "$PWD/src/docker/conf:/etc/nginx/conf.d"
      - "$PWD:/home/q/system/m_look_360_cn"
    ports:
      - "8082:80"
    container_name: "m.look.360.cn-nginx"
  php:
    image: "lizheming/php-fpm-yaf"
    volumes:
      - "$PWD:/home/q/system/m_look_360_cn"
    container_name: "m.look.360.cn-php"
```

```
server {  
    listen      80;  
    server_name dev.m.look.360.cn;  
    charset utf-8;  
  
    root /home/q/system/m_look_360_cn/public;  
    index index.html index.htm index.php;  
  
    error_page 500 502 503 504 /50x.html;  
    location = /50x.html {  
        root html;  
    }  
  
    # pass the PHP scripts to FastCGI server listening on 127.0.0.1:9000  
    #  
    location ~ \.php$ {  
        fastcgi_pass php:9000;  
        #fastcgi_pass unix:/tmp/fcgi.sock;  
        fastcgi_index index.php;  
    }  
}
```

Demo 演示

DOCKER 相关



1. 基于 Docker 的 CI 持续集成和 CD 持续支付
2. 基于 Kubernetes, Docker Swarm 的集群弹性扩容

参考资料

1. [什么是 Docker ?](#)
2. [Docker 从入门到实践](#)
3. [Docker compose 详解](#)
4. [深入浅出Docker](#)
5. [Hulk 容器服务](#)



THANK YOU!